DASH-05: Engineering Dashboards

> **Series:** DASH — Dashboard Design & Building | **Notebook:** 5 of 7 | **Created:** March 2026 | **Last Updated:** 10/26/2026

Overview

Engineering dashboards are the deepest tier in the dashboard hierarchy. They serve developers, DBAs, and performance engineers who need to answer specific questions: Which endpoint is slow? Which database query is causing timeouts? Did the last deployment improve or degrade latency? This notebook covers trace analysis, database performance, endpoint-level metrics, deployment impact analysis, and effective use of filters and variables for drill-down workflows.

Table of Contents

1. Span Analysis for Service Deep-Dives
2. Database Performance
3. Endpoint-Level Metrics
4. Deployment Impact Analysis
5. Code-Level Metrics
6. Engineering Dashboard Layout
7. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail — the Dashboards app and DQL are not available on Dynatrace Managed, which keeps classic dashboards
Permissions	`storage:spans:read`, `storage:logs:read`, `storage:metrics:read`
Data	Distributed trace data (spans), database spans, deployment events
Prior Reading	DASH-01 through DASH-04

1. Span Analysis for Service Deep-Dives

Engineering dashboards rely heavily on span data. Spans provide the most granular view of service behavior — individual requests, their duration, status, and attributes.

Every query in this section selects a transaction kind by hand, with `filter span.kind == "server"`. That is how you write the *tile* query, and it does not change. What changed is the exploratory pass that comes before it.

> **Forthcoming/rolling out (SaaS 1.344): the Services app filters by transaction kind and technology stack.** Published 07/27/2026, staged tenant rollout from 07/29/2026 — verify whether it has reached your tenant. The Services app gains filtering by **transaction kind** (HTTP, RPC, Messaging, Database) and by technology stack, plus support for primary Grail fields and tags as filter dimensions.
>
> The practical effect is on the scoping round-trip, not on the tile. Narrowing to "the messaging services in this stack" used to mean writing a query, reading it, adjusting the filter, and re-running — before you knew which services you were even building a tile for. Now that pass happens in the app: scope there interactively, confirm the shape of the result, then carry the confirmed filter into the tile query. `span.kind` remains how the tile expresses it either way, so nothing below needs rewriting — and if 1.344 has not reached your tenant, the DQL-first exploratory loop is unchanged and still works.

Service Throughput and Latency

```
// Service throughput and latency — engineering overview
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {request_count = count(), avg_ms = avg(duration) / 1ms, p50_ms =
percentile(duration, 50) / 1ms, p95_ms = percentile(duration, 95) / 1ms,
p99_ms = percentile(duration, 99) / 1ms, error_count =
countIf(span.status_code == "error")}, by:{dt.entity.service}
| fieldsAdd error_rate_pct = round(100.0 * error_count / request_count,
decimals: 2)
| sort p95_ms desc
```

Error Span Details

When investigating errors, engineers need the actual span attributes — status code, error message, and trace ID for correlation.

```
// Recent error spans with details — engineering investigation table
fetch spans, from:-1h
| filter span.kind == "server" and span.status_code == "error"
| fieldsKeep start_time, trace.id, dt.entity.service, http.route,
http.response.status_code, span.status_message, duration
| fieldsAdd duration_ms = duration / 1ms
| fieldsRemove duration
| sort start_time desc
| limit 25
```

Latency Distribution

Understanding the shape of latency distribution helps identify whether slowness is systemic or affects only a tail of requests.

```
// Latency distribution buckets — engineering histogram
fetch spans, from:-1h
| filter span.kind == "server"
| fieldsAdd duration_ms = duration / 1ms
| fieldsAdd latency_bucket = if(duration_ms &lt; 100, then: "&lt;100ms", else:
if(duration_ms &lt; 500, then: "100-500ms", else: if(duration_ms &lt; 1000,
then: "500ms-1s", else: if(duration_ms &lt; 5000, then: "1s-5s", else:
"&gt;5s"))))
| summarize request_count = count(), by:{latency_bucket}
| sort request_count desc
```

2. Database Performance

Database spans reveal query performance, connection issues, and which databases are bottlenecks.

Database Response Time by System

```
// Database response time by system and namespace
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(db.system)
// Connection opens, commits and driver phases are DB spans but not queries
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))
| summarize {avg_ms = avg(duration) / 1ms, p95_ms = percentile(duration, 95) /
1ms, query_count = count(), errors = countIf(span.status_code == "error")},
by:{db.system, db.namespace}
| fieldsAdd error_rate = round(100.0 * errors / query_count, decimals: 2)
| sort p95_ms desc
```

Slowest Database Operations

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and every one of them is null on Grail spans. They fail
SILENTLY: a filter on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Slowest database operations — engineering deep-dive
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(db.system)
// Connection opens, commits and driver phases are DB spans but not queries
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))
| summarize {avg_ms = avg(duration) / 1ms, max_ms = max(duration) / 1ms,
call_count = count()}, by:{db.system, db.query.text}
| sort max_ms desc
| limit 15
```

Database Query Trend

```
// Database query volume and latency trend
//
// Corrected 08/12/2026: `makeTimeseries` takes a BARE aggregation — dividing
inside it
// (`percentile(duration, 50) / 1ms`) fails with "the parameter has to be an
expression-based
// timeseries aggregation". Aggregate first, convert afterwards in
`fieldsAdd`. Note the unit
// conversion also changes: makeTimeseries yields a numeric array in
NANOSECONDS, not a duration,
// so `/ 1ms` silently yields null on the array — divide by 1000000.0 instead.
fetch spans, from:-2h
| filter span.kind == "client" and isNotNull(db.system)
// Connection opens, commits and driver phases are DB spans but not queries
| filter not(in(coalesce(db.operation.name, ""), {"CONNECT", "COMMIT",
"PREPARE", "RESULTSET"}))
| makeTimeseries {avg_dur = avg(duration), query_count = count()},
interval:5m, by:{db.system}
| fieldsAdd avg_ms = avg_dur[] / 1000000.0
| fieldsRemove avg_dur
```

3. Endpoint-Level Metrics

Endpoint-level analysis lets engineers identify which specific API routes are problematic.

Endpoint Performance Table

```
// Endpoint performance breakdown — engineering detail table
fetch spans, from:-1h
| filter span.kind == "server" and isNotNull(http.route)
| summarize {requests = count(), avg_ms = avg(duration) / 1ms, p95_ms =
percentile(duration, 95) / 1ms, errors = countIf(span.status_code ==
"error")}, by:{http.route, http.request.method}
| fieldsAdd error_rate = round(100.0 * errors / requests, decimals: 2)
| sort p95_ms desc
| limit 20
```

Endpoint Latency Trend

```
// Endpoint latency trend — line chart for the top 5 routes by average p95
//
// Corrected 08/12/2026: `makeTimeseries` takes a BARE aggregation — dividing
inside it
// (`percentile(duration, 50) / 1ms`) fails with "the parameter has to be an
expression-based
// timeseries aggregation". Aggregate first, convert afterwards in
`fieldsAdd`. Note the unit
// conversion also changes: makeTimeseries yields a numeric array in
NANOSECONDS, not a duration,
// so `/ 1ms` silently yields null on the array — divide by 1000000.0 instead.
fetch spans, from:-2h
| filter span.kind == "server" and isNotNull(http.route)
| makeTimeseries p95 = percentile(duration, 95), interval:5m, by:{http.route}
| fieldsAdd p95_ms = p95[] / 1000000.0
| fieldsRemove p95
// Rank on a scalar, not the array — sorting the array compares element by
element
| fieldsAdd p95_avg = arrayAvg(p95_ms)
| sort p95_avg desc
| limit 5
| fieldsRemove p95_avg
```

4. Deployment Impact Analysis

Compare service performance before and after a deployment to quantify its impact.

Before vs After Deployment

This pattern uses `append` to compare two windows. As written it compares the last 2 hours with the 2 hours before them, whenever it runs — it does not know when you deployed. To compare around a specific deployment, replace the window bounds with the deployment time: `from:toTimestamp("<deploy time>"), to:toTimestamp("<deploy time>") + 2h` for *after*, and `from:toTimestamp("<deploy time>") - 2h, to:toTimestamp("<deploy time>")` for *before*. Wrap the time in `toTimestamp()` — a bare string still works but is deprecated.

> **Next question after "did this release hurt?" is usually "what was in it?"** Once a tile has quantified a deployment's impact, the reader wants the tickets in that release — which is a deep link, not another query. **DASH-06 §7** builds a clickable per-release link from dashboard variables (`concat()` + the Markdown column type), so the release row in a deployment-impact dashboard can jump straight to the matching Jira / GitHub / GitLab / ServiceNow view. **Prerequisite:** the release-identifying process tags (`DT_RELEASE_STAGE` , `DT_RELEASE_PRODUCT` , `DT_RELEASE_VERSION`) with slug-safe values — without them there is nothing to parameterize the URL with. See FAQ-02 for the tagging mechanics.

```
// Before/after deployment comparison — last 2 hours vs preceding 2 hours
fetch spans, from:-2h
| filter span.kind == "server"
| summarize after_avg_ms = avg(duration) / 1ms, after_p95_ms =
percentile(duration, 95) / 1ms, after_error_rate = 100.0 *
countIf(span.status_code == "error") / count()
| append [
    fetch spans, from:-4h, to:-2h
    | filter span.kind == "server"
    | summarize before_avg_ms = avg(duration) / 1ms, before_p95_ms =
percentile(duration, 95) / 1ms, before_error_rate = 100.0 *
countIf(span.status_code == "error") / count()
  ]
```

5. Code-Level Metrics

When services are instrumented with custom spans, engineers can see method-level performance.

Span Performance by Code Namespace

```
// Code-level performance by namespace and function
fetch spans, from:-1h
| filter isNotNull(code.namespace)
| summarize {avg_ms = avg(duration) / 1ms, call_count = count()}, by:
{code.namespace, code.function}
| sort avg_ms desc
| limit 20
```

Span Hierarchy — Call Chain Analysis

```
// Span breakdown by kind — understanding the call chain
fetch spans, from:-1h
| summarize {span_count = count(), avg_ms = avg(duration) / 1ms}, by:
{span.kind}
| sort span_count desc
```

6. Engineering Dashboard Layout

Recommended Sections

Section	Tiles	Notes
Service Overview	Throughput table, error rate, latency percentiles	Use variables for service selection
Endpoint Detail	Endpoint performance table, latency trend	Filter by HTTP route
Database	DB response time table, query trend	Filter by db.system
Code Level	Namespace/function performance	Only if custom instrumentation exists
Deployment	Before/after comparison, recent events	Correlate with latency changes

Engineering Dashboard Best Practices

- **Use variables extensively** — service, namespace, and time range selectors let engineers focus on their area
- **Include trace links** — table tiles with `trace.id` let engineers click through to the full distributed trace
- **Short time ranges** — 15m to 1h for investigation; longer ranges hide the signal in noise
- **Table tiles over charts** — engineers often need exact values, not visual trends
- **Document queries** — add markdown tiles explaining what each section shows and how to interpret it

7. Summary and Next Steps

In this notebook you learned:

- Span-based service analysis with throughput, latency percentiles, and error rates
- Database performance monitoring using client spans
- Endpoint-level metrics for API route analysis
- Before/after deployment comparison patterns
- Code-level metrics for instrumented services
- Layout recommendations and best practices for engineering dashboards

Next: In **DASH-06: Variables and Filters**, we cover how to make dashboards dynamic and reusable with variables, filter propagation, and template patterns.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.